Applied Software Development Lab
Lab Handout 2: Blinking LED

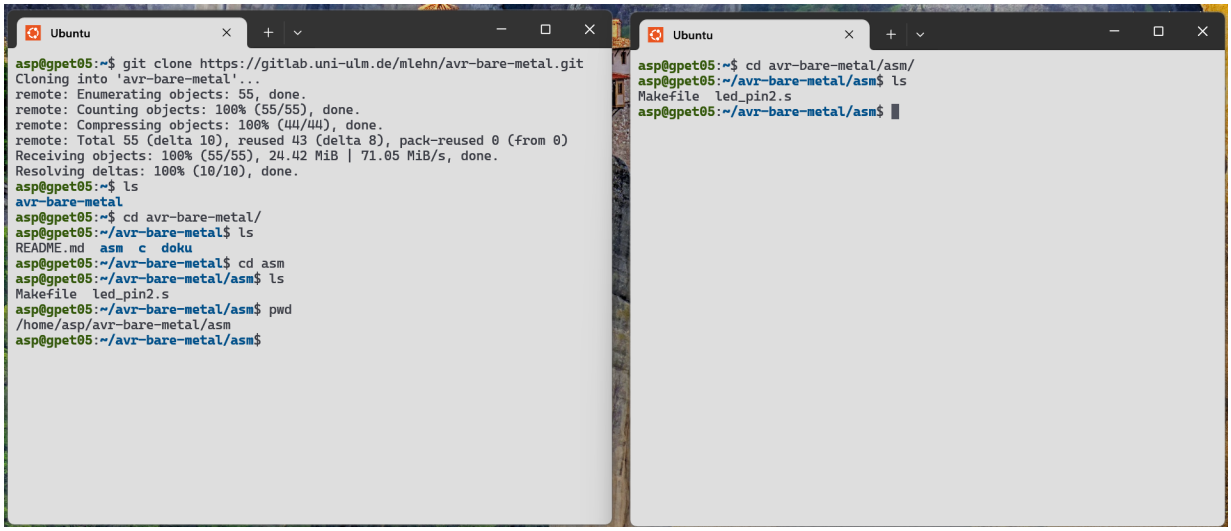
Learning objectives

In the last lab handout, you created machine code by hand. Today we show how the same machine code can be generated automatically from an assembly program – and then flashed to the microcontroller. You will also practice working with the terminal.

Step 1: Working with the terminal

Goal: You will learn to find your way around the terminal, to clone a repository with Git, and to navigate the file system.

We start with two terminals side by side – one for editing, one for running commands.



Left terminal (editing):

A) **Show the directory contents**
With the command `ls`, you can list the files and subdirectories in the current directory. Since it should be empty, nothing is displayed.

B) **Clone the Git repository**
`git clone https://gitlab.uni-ulm.de/mlehn/avr-bare-metal.git` downloads the code base that we use for this lab handout.

C) **Show the directory contents again**
Enter `ls` again. The directory `avr-bare-metal` should now be displayed – on the lab computers in blue, because it is a directory. Regular files appear in black.

D) **Show the current directory**
With `pwd` (= print working directory), you can see which directory you are currently in. This should be `/home/asp` – your home directory. The directory asp is a subdirectory of `home`, which in turn is located directly in the root directory `/`.

E) **Change into the subdirectory**
With `cd avr-bare-metal`, you change into the new directory. Use `pwd` to check that you are now in `/home/asp/avr-bare-metal`, and list its contents with `ls`. You should see a subdirectory `asm`.

F) **One directory deeper and opening a file**
Now change into the subdirectory with `cd asm` and check again with `pwd` and `ls` that you are now in `/home/asp/avr-bare-metal/asm`. You should see the file `led_pin2.s` – the assembly program. Open it in the editor with `nano led_pin2.s`.

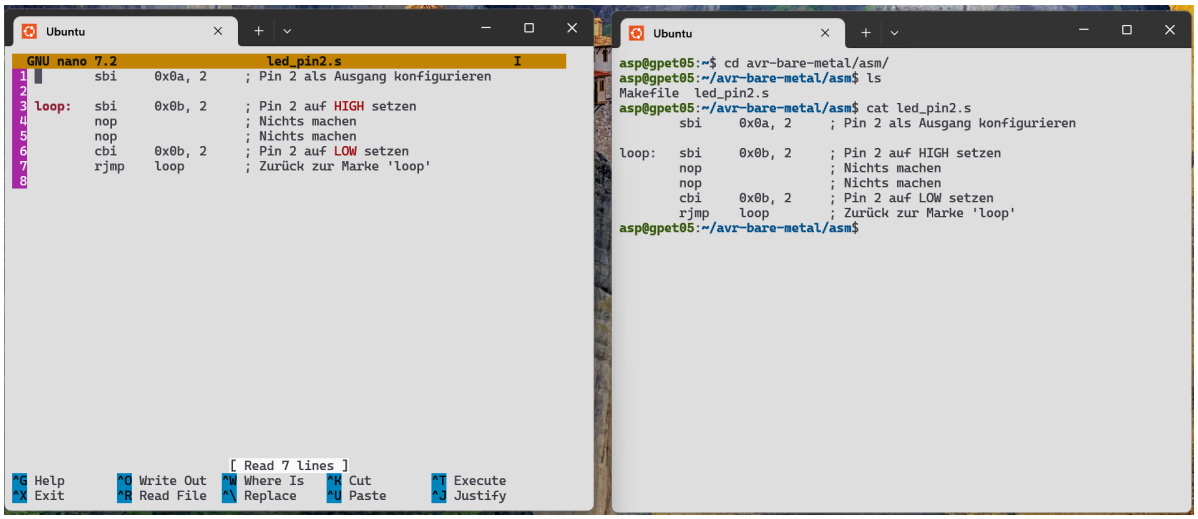
Right terminal (compiling & flashing)

A) **Change into the same directory**

The challenge: in the right terminal, you have to change manually into the same directory as in the left terminal. You can do this in one step with `cd avr-bare-metal/asm`, going down two directories at once. Alternatively, you can first run `cd avr-bare-metal` and then `cd asm`.

B) **Display the content of the file**

With `cat led_pin2.s`, you can display the content of the file – this is assembly code, a human-readable form of machine code. With the right tools, it can be turned into a so-called hex file that can be transferred to the microcontroller.



C) **Understand the Makefile and compile**

The so-called Makefile is also located in the current directory. It contains instructions on how the HEX file is generated automatically from the assembly program. Display it with `cat Makefile`. Then start the build process with `make`.

```
asp@gpet05:~/avr-bare-metal/asm$ make
avr-as -o led_pin2.o led_pin2.s
avr-ld -mavr5 -o led_pin2.elf led_pin2.o
avr-objcopy -O ihex -R .eeprom led_pin2.elf led_pin2.hex
rm led_pin2.elf led_pin2.o
asp@gpet05:~/avr-bare-metal/asm$
```

If everything works, a file `led_pin2.hex` is generated. It contains the machine code that you created by hand in the last lab handout. Look at its content with `cat led_pin2.hex`; it should look exactly like this:

```
:0C000000529A5A9A000000005A98FBCF58
:00000001FF
```

This is exactly the machine code from Lab Handout 1 that switches pin 2 between HIGH and LOW every 250 ns.

D) **Flash the microcontroller**

1. Connect the microcontroller via USB.
2. Run `usb_check`. This command makes sure that the WSL guest system can see the USB port.
3. Start the upload with `make upload HEX=led_pin2.hex`

E) **Measure with the oscilloscope**

Use an oscilloscope and measure the signal at pin 2 of the microcontroller. You should see a square wave with 250 ns HIGH and 250 ns LOW.

© Michael Lehn | CC BY-SA 4.0

Command overview for navigating the file system

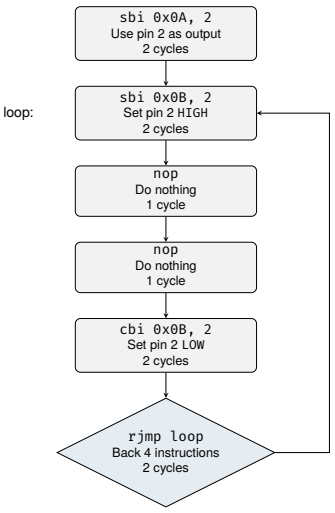
Command		Explanation
pwd	print working directory	Shows where you are in the file system
ls	list directory content	List of all files and subdirectories in the current directory
cd	change directory	Always changes into your own home directory
cd ..		Changes one directory up.
cd foo		Changes into the subdirectory foo (if it exists)
cd foo/bar		Changes into the sub-subdirectory foo/bar (if it exists)
cat foo.txt	concatenate files and print on the standard output	If the file foo.txt exists in the current directory, its content is printed. Handy for quickly checking what is in a file.

Step 2: A second look at the assembly program

In this task, you will work out how the assembly program is structured – and why it does the same thing as the machine code program that you wrote by hand in the last lab session.

```
1           sbi     0x0a, 2     ; 2 cycles
2
3 loop:     sbi     0x0b, 2     ; 2 cycles
4           nop                 ; 1 cycle
5           nop                 ; 1 cycle
6           cbi     0x0b, 2     ; 2 cycles
7           rjmp    loop        ; 2 cycles
```

Each line contains one machine instruction in symbolic form – this is assembly language. The instruction is on the far left, followed by its arguments, if any. Everything after `;` is a comment; here it states how many clock cycles the instruction takes.



Flowchart

The instructions in detail

Instruction	Meaning	Effect in the program
`sbi`	Set Bit in I/O Register	`sbi 0x0a, 2` Set pin 2 as output `sbi 0x0b, 2` Set pin 2 HIGH
`cbi`	Clear Bit in I/O Register	`cbi 0x0b, 2` Set pin 2 LOW
`nop`	No Operation	No effect, used to stretch the timing
`rjmp loop`	Relative jump to a label	Jumps back to the label `loop:`

Labels such as `loop:` serve as jump targets. The instruction `rjmp loop` makes the program continue at line 3 – this creates an endless loop. The flowchart shows the sequence over time graphically. It can help you understand how this program produces a square wave.

Task
Modify the assembly program so that a square wave with 500 ns HIGH and 500 ns LOW is generated at pin 2. As a reminder: one clock cycle takes 62.5 ns.

© Michael Lehn | CC BY-SA 4.0

Step 3: Blinking more elegantly with exclusive OR

The following program – as before – makes sure that a symmetric square wave is generated at pin 2: 250 ns HIGH and 250 ns LOW, alternating.
The difference lies in the code: this time we use registers and a new instruction – the exclusive OR (`eor`) in line 5.

```
1           ldi     r19,    4           ; 1 cycle
2           out     0x0a,   r19         ; 1 cycle
3           ldi     r18,    0           ; 1 cycle
4   loop:
5           eor     r18,    r19         ; 1 cycle
6           out     0x0b,   r18         ; 1 cycle
7           rjmp    loop                ; 2 cycles
```

Before we analyze what exactly the code does, we first test whether it really does what it promises.

Tasks

A) In the editor `nano` (left terminal), save your current file with **Ctrl + O** (or **Ctrl + S**) and exit the editor with **Ctrl + X**.
B) Create a new file with `nano led_pin2_xor.s`
C) Type in the program shown above and save it.
D) Assemble and flash the new program with `make upload HEX=led_pin2_xor.hex`
E) Check with the oscilloscope whether pin 2 switches between HIGH and LOW as claimed.

Explanation of the program (and one more task)

This program uses registers and an XOR operation (`eor`) to toggle the state of pin 2. Registers are small storage cells in the processor. The ATmega328P has 32 registers (r0 to r31), each 8 bits wide. Unlike normal RAM, they can be used directly in arithmetic operations. Besides addition and subtraction, an exclusive OR is also possible. The following describes what the individual lines do.

Line 1: `ldi r19, 4`
Writes the number 4 into register `r19`. Afterwards, `r19 = b00000100`, i.e., only bit 2 is set.

Line 2: `out 0x0a, r19`
Configures pins 0 to 7 of the microcontroller as inputs or outputs. Since only bit 2 is set in `r19`, only pin 2 is enabled as an output.

Line 3: `ldi r18, 0`
Initializes `r18` with 0, i.e., all bits are cleared.

Line 5: `eor r18, r19`
Combines `r18` with `r19` using an exclusive OR. This switches bit 2 in `r18` – this is also called “toggling”: if it was 0 before, it becomes 1; if it was 1 before, it becomes 0.
The content of `r18` therefore alternates between `b00000000` and `b00000100`.

Line 6: `out 0x0b, r18`
Outputs the content of `r18` to pins 0 to 7 of the microcontroller.
If a bit in `r18` is set and the corresponding pin is configured as an output, this pin is set HIGH. If the bit is not set, the pin (provided it is configured as an output) is set LOW.

Task: Explain to us why the exclusive OR in line 5 “toggles” bit 2.

© Michael Lehn | CC BY-SA 4.0

Step 4: Simple delay with one register

Compared with the previous program, only lines 7 to 10 are new in this example. So you do **not have to type in the entire program again**. As you will measure in a moment, these additional lines make pin 2 switch between HIGH and LOW more slowly.

```
 1          ldi     r19,    4       ; 1 cycle
 2          out     0x0a,   r19     ; 1 cycle
 3          ldi     r18,    0       ; 1 cycle
 4 loop:
 5          eor     r18,    r19     ; 1 cycle
 6          out     0x0b,   r18     ; 1 cycle
 7          ldi     r24,    0x02    ; 1 cycle (set counter for countdown)
 8 delay:
 9          subi    r24,    1       ; 1 cycle
10          brne    delay           ; 2 cycles if taken, 1 otherwise
11
12          rjmp    loop            ; 2 cycles
```

Tasks

1. Exit the editor. Then run the command `cp led_pin2_xor.s led_pin2_delay.s`. This gives you a copy of the file `led_pin2_xor.s` named `led_pin2_delay.s`.
2. Add lines 7 to 10 from the listing above to `led_pin2_delay.s`.
3. The figure on the next page shows a flowchart for the program. You can see that there are now two nested loops. The inner loop is new:
   (a) In line 7, register `r24` is initialized with the value `0x02` (i.e., `2`).
   (b) Lines 8 to 10 form the inner loop, in which `r24` is counted down until it has reached the value `0`. Only then is the loop exited.
   The project companion derives the following:

   If `r24` is initialized with **n**, one pass of the outer loop takes exactly **3·n + 4** clock cycles.

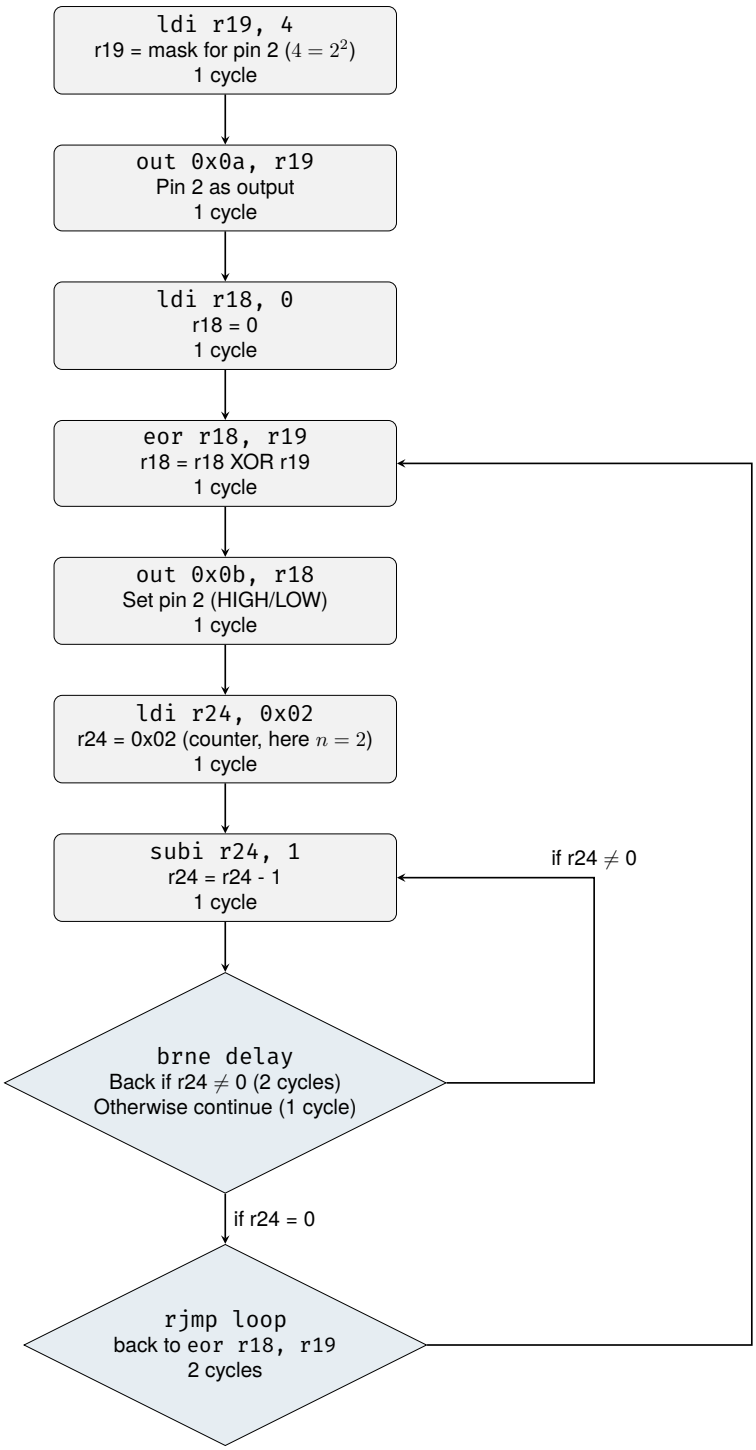
   This means: pin 2 stays HIGH for **3·n + 4** clock cycles and then LOW for just as many cycles before the state switches again.

   You will now verify this claim experimentally. To do so, fill in the table below. For different values of **n**, you
   - adjust the initialization of `r24` in line 7,
   - flash the program to the microcontroller again,
   - and then check with the oscilloscope how long pin 2 is HIGH and LOW.

Table for the measurements (1 clock cycle takes 62.5 ns)

n	3·n + 4	Exact duration for 3·n + 4 cycles	Measured HIGH duration	Measured LOW duration
1				
2				
3				
4				
255				
0				

© Michael Lehn | CC BY-SA 4.0



© Michael Lehn | CC BY-SA 4.0

Step 5: 32-bit delay with four registers

A single register of our microprocessor can only store values between `0x00` (i.e., 0) and `0xFF` (i.e., 255). That is not enough to delay the outer loop so much that the blinking of an LED connected to pin 2 can be perceived with the naked eye.

The following program therefore uses a 32-bit counter that is implemented with the four registers `r24` to `r27`. Together, these four registers form a so-called register group, which is treated like a single large register. The arrangement is:

```
r27 : r26 : r25 : r24
```

This group stores a 32-bit value in the so-called little-endian format – that is, the least significant byte is in `r24`, the most significant one in `r27`.

Example: if the registers are set as follows

```
r24 = 0x12, r25 = 0x34, r26 = 0x56, r27 = 0x78
```

then the combined value is:

```
n = r27:r26:r25:r24 = 0x78563412
```

In the program below, the registers are initialized as follows:

```
r24 = 0x00, r25 = 0x02, r26 = 0x00, r27 = 0x00
```

This corresponds to:

```
n = 0x00000200 = 512
```

That is: the inner loop runs 512 times.

```
 1          ldi     r19,    4           ; for pin 2 (4 = 2^2)
 2          out     0x0a,   r19
 3          ldi     r18,    0
 4 loop:
 5          eor     r18,    r19         ; 1 cycle
 6          out     0x0b,   r18         ; 1 cycle
 7
 8          ldi     r24,    0x00        ; 1 cycle
 9          ldi     r25,    0x02        ; 1 cycle
10          ldi     r26,    0x00        ; 1 cycle
11          ldi     r27,    0x00        ; 1 cycle
12 delay:
13          subi    r24,    1           ; 1 cycle
14          sbci    r25,    0           ; 1 cycle
15          sbci    r26,    0           ; 1 cycle
16          sbci    r27,    0           ; 1 cycle
17          brne    delay               ; 2 cycles if taken, 1 otherwise
18
19          rjmp    loop                ; 2 cycles
```

© Michael Lehn | CC BY-SA 4.0

Tasks

1. You do not have to type in the entire program again.
   Compared with `led_pin2_delay.s`, only lines 9 to 11 (initialization of the registers) and lines 14 to 16 (decrementing the register group) are new.
   ➜ Adjust your file `led_pin2_delay.s` accordingly.
2. The project companion explains in detail how to decrement a register group correctly. Here you are to verify the following claim experimentally (it only holds for $n > 0$):

   If `r24` is initialized with **n**, one pass of the outer loop takes exactly **6·n + 7** clock cycles.

   It follows that pin 2 stays HIGH for **6·n + 7** clock cycles and then LOW for just as long.

   Fill in this table (1 clock cycle takes 62.5 ns):

3. **Now calculate the value n at which pin 2 blinks at a rate of exactly 1 Hz.**

n	6·n + 7	Exact duration for 6·n + 7 cycles	Measured HIGH duration	Measured LOW duration
1				
2				

   For this to work, the pin must be HIGH for 500 ms and LOW for 500 ms. To do so, solve the equation:

   (6·n + 7) × 62.5 ns = 500 ms = 500000000 ns

   ➜ Calculate the matching value for n.
   ➜ If n is not an integer, round it to an integer.
   ➜ Initialize the counter accordingly, flash the program, and check with the oscilloscope whether the signal matches your expectation.
   ➜ Additionally, connect an LED with a series resistor to pin 2 and watch it blink at 1 Hz – hopefully pleasantly slowly and evenly.

© Michael Lehn | CC BY-SA 4.0